

Project #3: Concurrent Stock Server

Today: May 14, 2026
Submission Due: May 31, 2026
(Late submission: June 3, 2026, -10% per day)

1 Introduction

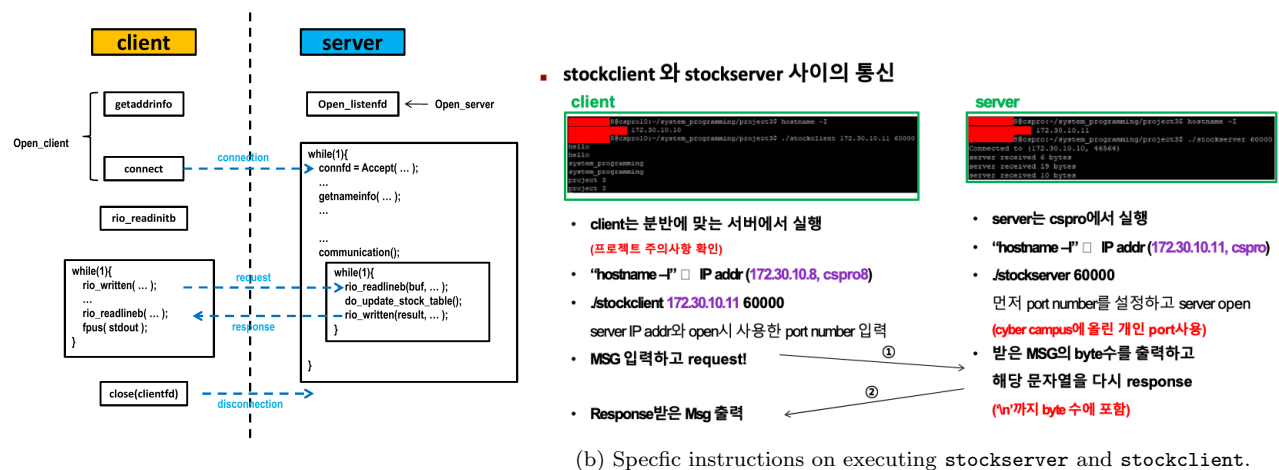
In this project, the students will apply the ideas of concurrency covered in the class. Students will learn this by programming a simple network server and host that simulates a mock stock exchange system, which is capable of serving multiple users. To represent these users, students will also program a client interface, which allows for interaction with the stock server.

Project Prerequisites. Familiarity with C/C++ programming and basic network programming concepts.

This project consists of three different tasks, with two tasks involving different approaches to the same problem, and the final task a performance analysis of both approaches.

2 Background

To provide background information about the inner workings of a server-client system, the code of a sample echoserver, along with single-client and multi-client is given. In the project files, once you compile the code with `make`, three binary executable will be created: `stockserver`, `stockclient`, and `multiclient`. `stockserver` and `stockclient` are the same as `echoserver` and `echoclient` as seen in class. It is recommended to try running `stockserver` and `stockclient` first, before running `stockserver` and `multiclient`.



(a) Diagram of interaction between the client and server.

Figure 1: Overview of the interaction between the client and server.

3 The Task: Concurrent Stock Server.

A single-process/thread server cannot serve multiple client connection and their requests at the same time, as all requests from each connected clients will be serialized and processed one at a time in such a scenario. To

serve multiple clients, the server needs to run make use of concurrent programming based on the lessons taught in class. To enable said concurrency, students will implement a concurrent stock server with the following two approaches in separate tasks.

1. An event-based approach using `select()`
2. A thread-based approach using the `pthread` library.

3.1 Explanation of Client

The client has four functions: `show`, `buy`, `sell`, and `exit`.

- `show`: Show the current available stocks in the market.
- `buy [stock ID] [amount of stock]`
- `sell [stock ID] [amount of stock]`
- `exit`: Leave the stock market server.

For this assignment, **only the stock server needs to be implemented**. The grading will be performed using the provided `multiclient.c` code. Also, note that for evaluation of your program, **no exception handling for unexpected inputs will be considered**. This includes typos and incompatible inputs. Information about what stocks and amount of stocks that a client has is also not considered. Thus, the `sell` command will never fail.

3.2 Implementation of Server

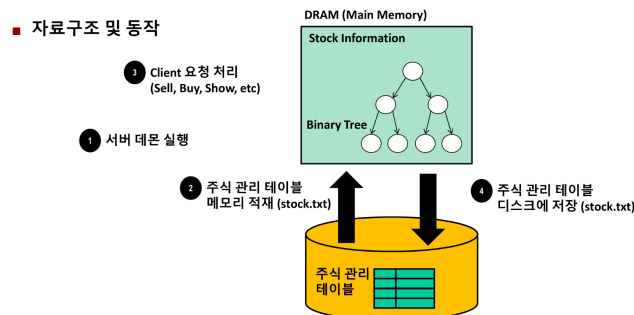


Figure 2: Overview of the inner workings of the stock server.

The server maintains the stocks in a text file named `stocks.txt` in a table format. Each row represents a stock entry, with three attributes:

- The ID of the stock. Each stock is given a random unique ID between 1 and `INT_MAX`.
- Remaining amount of stock.
- The price of the stock.

```

stock.txt
gr120210196@cspro8:~/my_project$ cat stock.txt
1 7 1000
5 3 3700
3 10 1200
4 8 5000
2 6 20000
  
```

Figure 3 shows an example of a `stock.txt` file. The file contains a table with three columns: ID, Amount, and Price. The first four rows are: 1 7 1000, 5 3 3700, 3 10 1200, 4 8 5000. The fifth row is 2 6 20000, which is highlighted with a red circle. Red arrows point from the text "ID = 2", "잔여 주식 = 6개", and "주식 단가 = 20000원" to the corresponding values in the highlighted row.

Figure 3: Example of a `stock.txt` file.

The following assumptions are made involving the status of the stocks.

- The price of each stock stays the same. Therefore, the only change is in the amount of stock that is left.
- If a request from a client requests to buy more stock than there is currently, then only the message **Not enough left stocks** is returned, and the request is not fulfilled.

To efficiently manage stock data, a binary tree can be used to hold the information of each stock. Each node will contain the previously mentioned three attributes (Stock ID, remaining amount, stock price), as well as a mutex to ensure a client exclusive access to said stock.

When the server starts, the server should read the contents of `stock.txt`, store it in the binary tree, and then start accepting requests from clients. **When the server is terminated via SIGINT, the contents of `stock.txt` needs to be updated with the new values for each stock from the binary tree.** Grading will be done based on whether if the values in `stock.txt` are properly updated. Note that the amount of concurrent clients and stocks may change from the given `multiclient.c` values.

In regards to synchronization and resource contention, while one client is reading a stock node i , another client cannot be allowed to write data in said stock node i . Therefore, students need to consider the readers-writers problem solution covered in class to achieve fine-grained locking with the mutex.

4 Task I: Event-driven approach

(Points: 30)

In this approach, each client triggers a file descriptor `fd`, and the server monitors the file descriptors with `select`. By monitoring multiple file descriptors in this way, synchronous I/O multiplexing and servicing of clients is achieved.

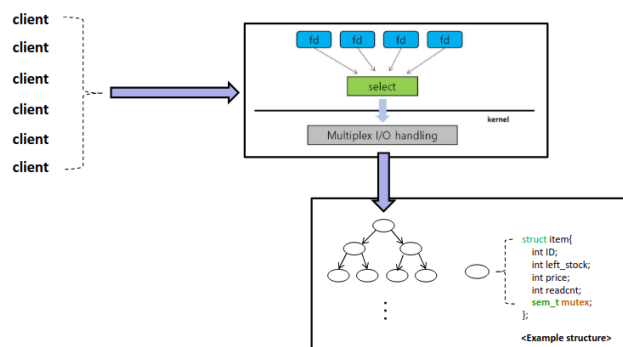


Figure 4: Overview of event-based approach of concurrent stock server.

5 Task II: Thread-based Approach

(Points: 30)

In this approach, a pool of worker threads is maintained to service incoming client requests. A master thread is kept running to accept initial connection requests from clients, and to insert descriptors to a intermediary buffer. Each worker thread in the pool then takes a descriptor and provides service to the client until the client exits the server.

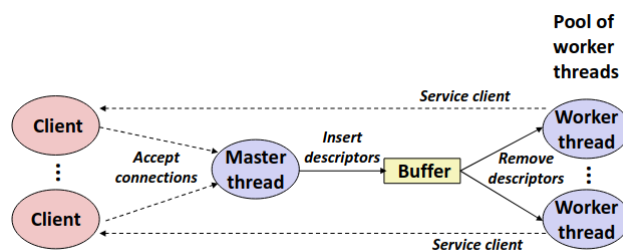


Figure 5: Overview of thread-based approach of concurrent stock server.

6 Task III: Performance evaluation and analysis (Points: 30)

The final task of the project is to evaluate the performance of both approaches. The analysis should be done by changing the configuration of the client count in `multiclient.c`, and compare the elapsed time between the two approaches based on change in client count. For this task, you can use more clients than in testing for previous tasks, such as 10 to 20 clients. You can use the `gettimeofday` function to make time measurements for elapsed time.

Some criteria/ideas to perform analysis on are as follows:

- Expandability: Analyze the effect of increasing client count has for each approach.
- Workload type: Analyze how the ratio of client requests types (buy, show, sell) affects performance.
- An analysis of whether or not your implementation of the two approaches match what you've learned in class. If the performance results don't match, why?

7 Appendix

7.1 More information on multiclient

`multiclient.c` allows for the evaluation of your concurrent stock server by creating a specified amount of client processes, and issue a specified amount of show, buy, and sell commands for each client.

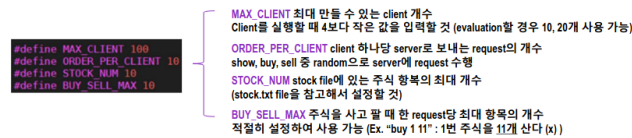


Figure 6: `#define` settings for `multiclient.c`.

To evaluate your server, do the following:

1. Set the `#define` values.
2. Start your stock server (`./stockserver 1119`)
3. Run `multiclient`. (`./multiclient [host server IP_addr] [port #] [client_num]`) During normal testing, remember to keep the `client_num` value to 4 or less.

7.2 Server output format

- `[buy] success`: When a buy request from a client is successful.
- `[sell] success`: When a sell request from a client is successful.
- `Not enough left stocks`: When there is not enough stock to fulfill a buy request.

8 Submission Guideline

Process Management (IMPORTANT): Make sure that no background processes that are not used continue to run while working on the project. Use the `ps -aux` or `top` commands to do so. If background processes do not terminate or if the `cspro` server does not work, please contact the TAs as soon as possible.

CSPRO server assignments (IMPORTANT): Ensure that you are using the correct servers for both the client and server that was assigned to your class, as well as the individual port that you have been assigned. Refer to the assignment page for more details on ports.

- Class 1 (CSE4100-01, AIE4050-01): Use `cspro8` for client, `cspro` for server.
- Class 2 (CSE4100-02, AIE4050-02): Use `cspro9` for client, `cspro` for server.

Hand-In Specifications: The submission should contain only source code file(s), `include` file(s), `Makefile`(s), the `stock.txt` file, and the performance report `.pdf` file. **All executable program files must be removed.** The TAs will automatically rebuild your shell program from the provided source code.

Please strictly follow the submission instruction below. If you not, you may get a penalty on your overall project score.

1. Only submit one archive file. The submitted archive file (`.tar`) must be named `prj3-your_student_id.tar`.
2. When the `tar` file is extracted, the name of the root directory of the tar file must be your student id.
 - **Points will be deducted(-20%)** if the extracted tar file is not organized as shown in the submission example below.
3. You must create directories for each phase with the name of `task1/2`.
4. Do not make changes or program your implementation in the `csapp.c` and `csapp.h` files. The TAs will automatically replace them with the original files. Failure in doing so will result in **no points being awarded**.
5. Each directory for each phase must include a `Makefile`, and source codes. You also have to submit one document for the performance evaluation and analysis task. Failure to use a `Makefile` will result in **no points being awarded**. You must provide a `Makefile` that properly compiles your code. No additional makefile submissions will be accepted after the due date.

Attachment File:

- Task I folder (source code(`csapp.c/h`, `echo.c`, `multiclient.c`, `stockclient.c`, `stockserver.c`), `Makefile`, `stock.txt`) (30 points)
- Task II folder (source code(`csapp.c/h`, `echo.c`, `multiclient.c`, `stockclient.c`, `stockserver.c`), `Makefile`, `stock.txt`) (30 points)
- Performance evaluation and performance report (30 points)

Following is an example of a submission.

```
$~> ls
prj3_20221234.tar
$~> tar -xvf prj3_20221234.tar
$~> ls
20221234 prj3_20221234.tar
$~> tree 20221234
20221234
├── document.pdf
├── task1
│   ├── Makefile
│   ├── csapp.c
│   ├── csapp.h
│   ├── echo.c
│   └── multiclient.c
```

```
|  — stock.txt
|  — stockclient.c
|  — stockserver.c
| task2
|  — Makefile
|  — csapp.c
|  — csapp.h
|  — echo.c
|  — multiclient.c
|  — stock.txt
|  — stockclient.c
|  — stockserver.c
```

All students are requested to upload the archived source files on Cyber Campus.

Note: Please make sure to compile your source code on CSPRO server, as the TA will build and compile your submitted project on that server. If the submitted code does not compile it cannot be scored!!!

9 Miscellaneous

9.1 Best practices the system programmers must comply to

WHAT YOU MUST DO:

1. You must read Chapter 12 (Concurrent Programming) of the textbook in its entirety, as well as the class slides related to said chapter, to fully understand, learn and complete this project.
2. Read the complete project specifications before programming your shell project.
3. The **Makefile** needs to include all dependencies (libraries etc) required to build your program. If your code does not compile after running **make** inside each task directory (i.e a compile error), you will not receive any marks for that task.
4. Source code commenting is a professional programmer's practice, and a must-do in this project as well.

WHAT YOU MUST NOT DO:

1. **Do not hand-in any binary or object code files in your source code directory.**
2. Do not include any hardcoded paths in your makefile.
3. **Do not copy or share your source code**, it is strictly prohibited otherwise you will be given penalty for such an action.